Check: Even OR Divisible by 5

Question 1: 2025 Jan OPPE 1 - Set 1

Problem Statement

Write a Function to Check Conditions

1 Function Signature

```
is_even_or_divisible_by_5(num)
```

2 Parameter

Integer `num` to evaluate

3 Return Value

True if even OR divisible by 5

Otherwise **False**

Important: This is a function-only question. Do not take input or print output—just complete the function.

Understand the OR Operator

Key Difference: OR vs AND Logic

OR Operator

At least one condition must be true

Either condition passes ↣ return True

Both fail ↣ return False

AND Operator

Both conditions must be true

One fails ↣ immediately False

Rarely used in multiple conditions

The Two Conditions

Condition 1: Even Number

A number is even when it is divisible by 2, i.e., when division by 2 leaves no remainder.

```
num % 2 == 0
```

- If remainder is 0 ↣ **even**
- If remainder is 1 ↣ **odd**

Condition 2: Divisible by 5

A number is divisible by 5 when division by 5 leaves no remainder.

```
num % 5 == 0
```

- If remainder is 0 ↣ **divisible**
- If remainder ≠ 0 ↣ **not divisible**

Example Table

Understanding True & False Cases

Input	Even?	Divisible by 5?	Final Result
8	Yes ✓	No	**TRUE**
10	Yes ✓	Yes ✓	**TRUE**
15	No	Yes ✓	**TRUE**
7	No	No	**FALSE**

> **Key Insight:** Unlike AND questions, here passing ONE condition is enough to return True.

Many students confuse this in exams—be careful!

Decision Logic

Think Through the Steps Clearly

Step 1: Check if Even

Evaluate `num % 2 == 0`. This determines if the number is divisible by 2 without remainder.

Step 2: Check if Divisible by 5

Evaluate `num % 5 == 0`. This checks for divisibility by 5.

Step 3: Apply OR Logic

If either condition is True ↣ return True. If both are False ↣ return False.

Key Point

No need for nested if statements! Keep it simple and direct.

Short-Circuit Logic

Python's Optimization Strategy

1

Left to Right Evaluation

Python checks conditions in order: first left, then right.

2

First Condition True?

If first condition passes ↣ second condition is **not even checked**.

3

First Condition False?

Only then does Python check the second condition.

Short-circuit evaluation makes your code faster and more efficient—especially important when conditions involve expensive operations.

The Elegant Solution

One Line of Clean, Professional Code

The Complete Function

```
def is_even_or_divisible_by_5(num:
int) -> bool:
    return num % 2 == 0 or num % 5
== 0
```

That's it.

- No loops required
- No extra variables
- No overthinking needed

Naive vs Clean Approach

Common Mistakes vs Professional Code

❌ What Many Students Do

Overcomplicated

Write multiple if-else blocks when unnecessary

Repetitive

Repeat return statements multiple times

Nested

Create deep nesting that's hard to debug

Confusing

Make simple logic unnecessarily complex

```
if num % 2 == 0:
    return True
else:
    if num % 5 == 0:
        return True
    else:
        return False
```

Too much code for the same logic.

✅ What You Should Do

Flattened Logic

Use boolean expressions directly

Concise

One return statement, clear intent

Efficient

Less code means fewer bugs

Professional

Industry-standard clean coding style

```
return num % 2 == 0 or num % 5 == 0
```

Cleaner, faster to write, and less prone to errors.

Engineering Best Practices

1. Understand Logical Operators

OR means at least one condition must pass. Master the difference between OR and AND logic to avoid common mistakes.

2. Avoid Deep Nesting

Flat logic is easier to read, debug, and maintain. When you find yourself nesting more than 2 levels deep, reconsider your approach.

3. Use Boolean Expressions Directly

Don't write if-else to return True/False when the condition itself is boolean. Just return the condition expression.

Final Logic Pattern

```
Result = (Even) OR (Divisible by 5)
```

```
return num % 2 == 0 or num % 5 == 0
```